

JioPC × IIT Bombay

Open Innovation Challenges — Hackathon 2026

Partner: Institute Technical Council, IIT Bombay

Edition: May 2026

Challenge 01 — JioPC Home: Engaging Desktop Experience

Difficulty: Hard | Max points: 100

1. Background & Problem Statement

JioPC is more than a desktop — it is a gateway to everything computing makes possible. The challenge is to make the moment a user opens JioPC feel like nothing they have experienced before: a landing experience that is personal, alive, and immediately compelling. Something that draws users in, keeps them engaged, and makes them want to come back.

The challenge is to build the JioPC Desktop Home Screen & Menu — the primary desktop shell experience a user sees when they open JioPC. It brings together a fast, Mac-style dock; a rich application menu; live desktop widgets driven by a content engine; a theme engine for personalisation; and a first-time wizard that welcomes new users. The Desktop Home Screen & Menu should surface what JioPC can do, help users find apps and content relevant to them, and give them a fast path to their most common tasks — whether they are opening JioPC for the first time or returning for the hundredth time. Everything must stay lightweight enough to run on 4 vCPU with no GPU, and all preferences must persist in the user's home directory so they survive VM reassignment.

1.1 What to Build

The solution has five distinct engineering components.

Component A — Dock Bar

A macOS-style dock that is the primary surface for fast app access. It is always available, anchored to a screen edge (default: bottom), and stays lightweight without GPU compositing. The dock must:

- Show pinned / favourite apps, launched from their .desktop files on a single click
- Indicate which apps are currently running and let the user focus a running window from the dock

- Allow the user to pin, unpin, and reorder apps; the pinned set persists in the user's home directory
- Show hover labels / tooltips, and open the Application Menu (Component B) from a dock affordance
- Render smoothly on CPU only — any hover or magnification effect must stay within the CPU budget

Component B — Application Menu

A full application menu / launcher (opened from the dock or a hotkey) that lists every installed app and helps the user find the right one quickly. It must read installed apps from .desktop files in /usr/share/applications/ and ~/.local/share/applications/ and provide:

- A Recently Used section, tracked from a usage log in the user's home directory
- Category filters derived from the .desktop Categories= field
- Order-by controls — e.g. by name, most used, and recently used
- Live search across app name and description (the .desktop Comment= field)
- A Recommended Apps section that surfaces apps relevant to the user (e.g. apps they have not yet opened, prioritised by category relevance)
- Each app shown with icon, name, and one-line description; launch on click; usage persisted to the user's home directory

Component C — Desktop Widgets (Widget Engine)

A widget engine that renders live widgets on the desktop. The engine must be extensible — adding a new widget type should mean adding a plugin, not rewriting the shell. Widgets that show dynamic content fetch from a CMS: you define the content schema and ship a sample / mock CMS endpoint (JSON); JioPC fills the real content later. The widget engine must:

- Support a configurable set of widgets placed on the desktop — at minimum a featured/hero carousel, a news/headlines widget, quick-launch tiles, and a greeting/clock widget
- Let the user add, remove, and arrange widgets on the desktop, with the layout persisted to ~/.config/jiopc/home/layout.json in the user's home directory
- Fetch CMS content at session start and cache it locally in ~/.cache/jiopc/home/ in the user's home directory
- Serve cached content when the CMS endpoint is unreachable — no crash, no blank widget
- Support content refresh at a configurable interval (default: once per session)
- Document the CMS JSON schema as part of the deliverables

Component D — Theme Engine (Personalisation)

A theme engine that lets users personalise the look and feel of the desktop shell. This is an engine, not a fixed set of colour presets — it must allow the underlying theme attributes (colours, fonts, sizes, and similar styling properties) to be changed, and apply them consistently across the dock, application menu, and widgets. The theme engine must:

- Allow changing of theme attributes — colours, fonts, and other styling properties — through a settings surface
- Apply the active theme consistently across the dock, application menu, and widgets
- Support at least a light and a dark theme out of the box, built using the same engine
- Persist the active theme (and any user-defined theme) to the user's home directory so it survives VM reassignment

Component E — First-Time Greeting / Wizard

A guided onboarding experience that runs once, the first time a user logs in. It must:

- Show a personalised greeting using the user's name and the current time / date
- Walk the user through the key parts of JioPC — the dock, the application menu, desktop widgets, and theming
- Let the user make a few initial choices (for example, pick a theme and pin a few favourite apps)
- Run only once — a completion flag is persisted to the user's home directory so the wizard does not re-trigger on subsequent logins

Reference Design

Design reference screenshots are provided separately. These are for inspiration and directional guidance — they show one interpretation of the dock, application menu, desktop widgets, theme settings, and the first-time wizard. You are not required to match them exactly. The engineering requirements above define what must be built; the visual design is your decision.

2. Constraints

2.1 Common JioPC Constraints (apply to all challenges)

Your solution runs on the JioPC floating-VM platform. The following platform constraints apply to every challenge. (Full architecture is in Document 1.)

Platform constraint	Detail
Hardware profile	4 vCPU @ 2.45 GHz, 8 GB RAM, no GPU. All rendering is CPU-only.
No root at runtime	Runs as a user-space process — no sudo during normal operation.
Floating VM	A user may land on a different VM each login. No reliance on VM-local state.
Persistent home only	All user state must live in /home/user (NFS), which roams across all VMs.
Ships as a .deb	Delivered as a .deb baked into the Gold Image; must install cleanly on a fresh Ubuntu 24.04 + LxQt VM.
VM-tested	Built, benchmarked, and demonstrated inside an Ubuntu 24.04 + LxQt VM.

2.2 Challenge-Specific Hard Constraints

Constraint	Detail
------------	--------

No GPU	No hardware-accelerated compositing. The dock, menu, and widgets must render correctly with compositing disabled.
Idle CPU < 10%	Measured over 30 seconds with the dock, menu, and widgets loaded. Content fetch and cache must not cause sustained CPU spikes.
Idle RAM < 200 MB	Measured after 5 minutes idle, with all components loaded and content cached.
Session start to desktop shell visible	< 3 seconds from session login.
All preferences in home directory	Pinned apps, widget layout, theme, and usage history live in ~/.config/jiopc/home/ and ~/.local/share/jiopc/home/. No VM-local state.
Offline graceful degradation	If the CMS endpoint is unreachable, show cached content or an empty widget state. No crash.
Coexists with the LxQt session	Must not break the underlying LxQt session; the standard launcher / panel must remain reachable.
No root during operation	Runs as a user-space process — no sudo at runtime.

2.3 What We Are NOT Looking For

- A static wallpaper change or a static HTML page — this is a running desktop shell with live data, widget state, and persisted user preferences.
- Hardcoded widget content — all dynamic content must come from the CMS or from installed .desktop files.
- A heavy Electron app that consumes > 200 MB RAM at idle.
- A layout that looks good at 1920×1080 but breaks at 1280×720 over RDP.
- A dock, menu, widget layout, or theme that resets on every session — all preferences must persist in the user's home directory.

3. Tools Needed

The environment is fixed (Ubuntu 24.04 + LxQt, 4 vCPU, no GPU). Language and framework are your choice and must be justified in the design document. The recommendations below are guidance, not mandates — they are chosen to fit the no-GPU, < 200 MB RAM budget.

3.1 Recommended Stack

Area	Recommended	Why
UI framework	Qt 6 — C++/Qt Widgets or Python with PySide6 / PyQt6 (Qt Quick / QML optional for the carousel and animations)	Native and lightweight, integrates cleanly with LxQt (itself Qt-based), renders well without GPU compositing, and avoids the Electron RAM penalty.
Dock & app menu	Qt widgets; read .desktop files via PyXDG / GLib GKeyFile; running-window info via wmctrl / X11	Standard way to enumerate apps, categories, and running windows for the dock and menu.
Widget engine & CMS	A plugin architecture for widgets; Qt Network or Python requests / libcurl with a JSON parser for the CMS	Keeps widgets extensible and the CMS endpoint configurable; easy to fetch and cache.
Theme engine	Qt Style Sheets (QSS) and QPalette	Lets colours, fonts, and styling be

	for colours, plus font configuration	changed centrally and applied across dock, menu, and widgets.
Persistence	Plain JSON files under ~/.config/jiopc/home/ and ~/.cache/jiopc/home/	Survives VM reassignment via the NFS home; no database needed.
Autostart	XDG autostart .desktop entry shipped in the .deb	Launches the desktop shell at session start.
Benchmarking	hyperfine for start-up timing; /proc and ps for CPU/RAM	Reproducible, documented methodology for the performance metrics.

3.2 Reference Links

- Qt Documentation — <https://doc.qt.io/>
- LxQt Project — <https://lxqt-project.org/> | LxQt GitHub — <https://github.com/lxqt>
- Freedesktop .desktop / menu spec — <https://specifications.freedesktop.org/desktop-entry-spec/latest/>
- Flatpak / Flathub — <https://flathub.org/>
- XDG Autostart spec — <https://specifications.freedesktop.org/autostart-spec/latest/>
- hyperfine (benchmarking) — <https://github.com/sharkdp/hyperfine>

4. Evaluation Criteria

In addition to the metrics and checklist below, this challenge is scored on the common five dimensions — Functionality (30%), Performance (25%), Code Quality (20%), Documentation (15%), and Innovation/Bonus (10%) — defined in Document 1, Section 5.

4.1 Success Metrics

Metric	Target
Session login to desktop shell visible	< 3 seconds
Idle CPU (dock, menu, and widgets loaded)	< 10%
Idle RAM (all components loaded)	< 200 MB
Dock: pin / unpin / reorder, launch, running-app indicators	Required
Application menu: recently used, category filters, order-by, search, recommended apps	Required
Widget engine: add / remove / arrange widgets; content from CMS	Required
Offline: cached CMS content shown when endpoint unreachable	Required — no crash
Theme engine: change colours and fonts, applied across dock / menu / widgets	Required
All preferences persist across VM reassignment (pinned apps, layout, theme, usage)	100%
First-time wizard runs once on first login	Required
Renders correctly at 1280×720 over RDP	No layout breaks, no unreadable text
First-time user test (5 people, no instruction)	4 of 5 find and open an app within 60 seconds

4.2 Acceptance Checklist (EVALUATE)

Check	Pass?
.deb installs cleanly on fresh Ubuntu 24.04 VM ("sudo dpkg -i package.deb" without errors)	Yes / No
Desktop shell visible within 3 seconds of session login	Yes / No
Dock present — pin/unpin/reorder, launch apps, and show running-app indicators	Yes / No
Application menu — recently used, category filters, order-by, search, and recommended apps	Yes / No
Widget engine — add/remove/arrange widgets, and widgets render content from the CMS	Yes / No
Content fetched and displayed from the configurable CMS endpoint	Yes / No
Offline degradation — cached content or empty widget states, no crash	Yes / No
Theme engine — changing colours and fonts applies across dock/menu/widgets and persists	Yes / No
Preferences persist across simulated VM reassignment (fresh session, same home)	Yes / No
First-time wizard runs once and does not re-trigger on the next login	Yes / No
Idle CPU < 10% (30-second average, all components loaded)	Yes / No
Idle RAM < 200 MB (VmRSS after 5 minutes idle)	Yes / No
Renders at 1280×720 inside VM over RDP — no breaks or unreadable text	Yes / No
LxQt session remains functional / standard launcher reachable	Yes / No

5. Deliverables

5.1 What to Hand Over (SUBMIT)

Deliverable	Details
Desktop shell source	Complete source for the dock and application menu. Must build on Ubuntu 24.04. Language and framework open — justify your choice in the design document.
Widget engine + sample widgets	The extensible widget engine plus at least the minimum widget set (carousel, news, quick tiles, greeting/clock). Layout state saved to the user's home directory.
CMS content schema + mock CMS	Documented JSON schema for the CMS endpoint, plus a sample / mock CMS feed with at least 5 entries (carousel items, news, tiles) usable for testing.
Theme engine + sample themes	The theme engine with controls to change colours and fonts, plus at least a light and a dark theme built on it. Active theme persisted to the home directory.
First-time wizard	The once-only onboarding flow with greeting, feature tour, and initial choices; completion flag persisted to the home directory.
Design document (design.md)	Architecture of all five components, data flow diagrams, technology choices with justification, and known limitations.
.deb package	Installs the full desktop shell on Ubuntu 24.04 + LxQt without user interaction. Includes XDG autostart entry.
Benchmark report	Idle CPU, idle RAM, session-start-to-visible time. p50 and p95. Methodology documented.
Screenshots / recording	Desktop shell on first open (incl. the wizard), after customisation (pinned apps, arranged widgets), with a different theme applied, and in offline mode — at 1920×1080 and 1280×720 inside a VM.

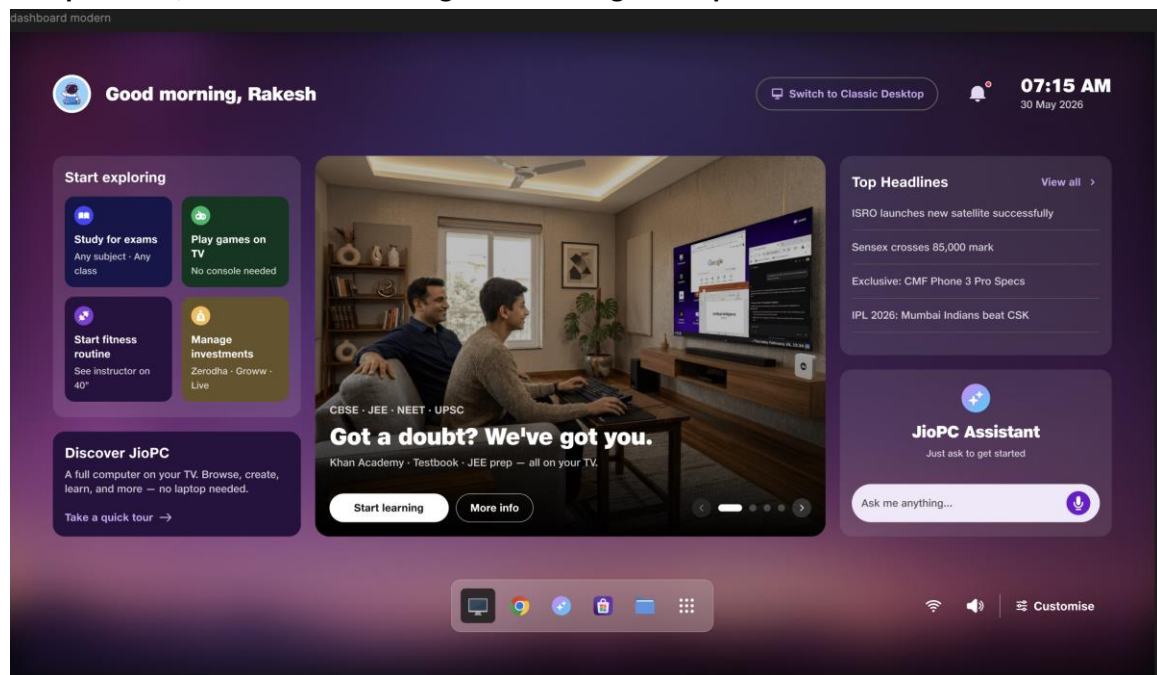
5.2 Bonus Goals (optional)

- Persona detection — infer user type from usage patterns and adapt the default dock, menu, and widgets.
- 'Try this today' widget — a daily rotating suggestion highlighting one JioPC feature the user has not used, with a one-click launch.
- Additional widget plugins beyond the minimum set, demonstrating the extensibility of the widget engine.
- Theme import / export — let users save and share custom themes.

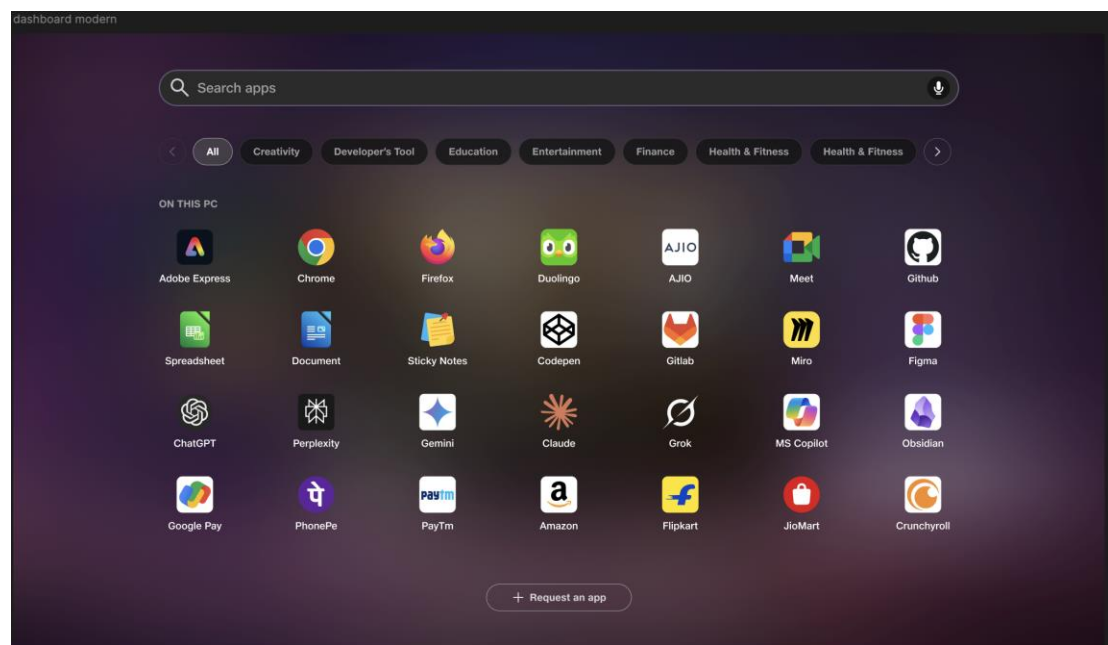
6 Appendix

- Reference (designs)

Component A, C: Dock bar and Widgets on landing desktop screen



Component B: Application Menu launched from icon dock bar on desktop landing screen



- Component C: Desktop Widget (library/ Engine)

